

KonnectStudios — Full Stack Developer Take-Home Project

Role: Junior–Mid Full Stack Web Developer **Expected time:** 1.5–2 hours for the core requirements (the bonus below is extra credit, not counted in this estimate) **Submission:** GitHub repo link (private, invite [your-github-username]) + short README

What to build

A **studio bookings list page** that reads from a small JSON dataset (provided below) and lets the user view, filter, and add bookings.

You're given this starting data — copy it into a file in your repo and treat it as your data source (no real backend/database required):

```
[
  { "id": 1, "clientName": "Mia Chen", "sessionType": "Podcast Recording",
    "date": "2026-07-02", "status": "confirmed" },
  { "id": 2, "clientName": "Jordan Reyes", "sessionType": "Photography", "date":
    "2026-07-04", "status": "pending" },
  { "id": 3, "clientName": "Aisha Patel", "sessionType": "Video Shoot", "date":
    "2026-06-28", "status": "cancelled" }
]
```

Requirements

1. **List view** — Display all bookings in a clear, well-organized layout (table or cards, your call). At minimum, this means:
 - A clear visual hierarchy — page title, then the list — so the structure is obvious at a glance.
 - Each booking's `status` is visually distinguishable, not just text (e.g. a color-coded badge/tag for confirmed/pending/cancelled).
 - Consistent spacing and alignment across rows/cards — no overlapping text, no inconsistent gaps.
 - Legible typography and enough contrast to read comfortably (default browser styling with zero CSS will not meet this bar).
2. **Filter** — Let the user filter the list by `status` (confirmed / pending / cancelled).
3. **Add a booking** — A small form to add a new booking to the list (client-side state is fine — it doesn't need to persist on refresh).
4. **Validation** — The add form should reject empty fields and a date in the past, with a clear error shown to the user.
5. **Empty state** — If a filter produces zero results, show something better than a blank page.

Use plain HTML/CSS/JS, or React/Vue if you prefer — your choice, no framework is mandated.

Bonus (optional, not required to pass)

Want to go further? Wire up a **real, free scheduling tool** instead of (or alongside) the "Add a booking" form — for example, embed a free [Calendly](#) inline widget or pop-up, or an open-source alternative like [Cal.com](#), so a visitor could book an actual time slot.

A few notes if you attempt this:

- This is a genuine bonus — candidates who skip it are evaluated only on the core requirements above, and a clean core submission beats a rushed bonus.
- No backend or API key handling is expected: a client-side embed (e.g. Calendly's `data-url` widget script) is enough to demonstrate the integration. Don't spend time trying to sync embedded bookings back into the JSON list — treat them as two separate, clearly labeled sections of the page if you include both.
- Calendly's basic embed is free on all plans and needs a Calendly account + scheduling link, but no API key, for the simple inline/pop-up embed. If you'd rather not create an account, Cal.com's embed works similarly and is fully open source.
- Mention in the README that this was a stretch goal and roughly how long it took, so it's clear it didn't eat into time budgeted for the core requirements.

What we're actually evaluating

This task is intentionally small. What we're looking at is **not** whether you can build a bookings list — it's:

- **How you organize things** — file/folder structure, component boundaries, naming. Would another developer understand this in 2 minutes?
- **Visual organization** — does the page read as deliberately laid out (hierarchy, spacing, alignment, status cues), or like unstyled markup? This is about basic clarity, not aesthetic polish — see below.
- **Level of detail** — did you handle the empty state, the validation error, the "no results" case, or just the happy path?
- **Consistency** — does the UI/code follow a clear pattern throughout, or does it feel ad-hoc?
- **Communication** — your README should briefly explain how you structured things and any trade-offs you made under the time limit.

What we are not evaluating

- Backend, deployment, or persistence
- Visual **polish** — custom branding, illustration, animation, or pixel-perfect design (basic layout/spacing/hierarchy from Requirement 1 is still expected; this line is about going beyond that, not skipping it)
- Test coverage
- Using a particular framework or library

Submission checklist

- ☐ GitHub repo (private), [your-github-username] invited as collaborator
- ☐ README: how to run it locally, and a few lines on how you organized the code and why

If anything here is ambiguous, make a reasonable call, note your assumption in the README, and move on.